

Searching for the minimum

A genetic algorithm for the 2D Ackley function

From random points to a solution near (0, 0).

PYTHON + WEB VISUALIZER

A visual, reproducible walkthrough

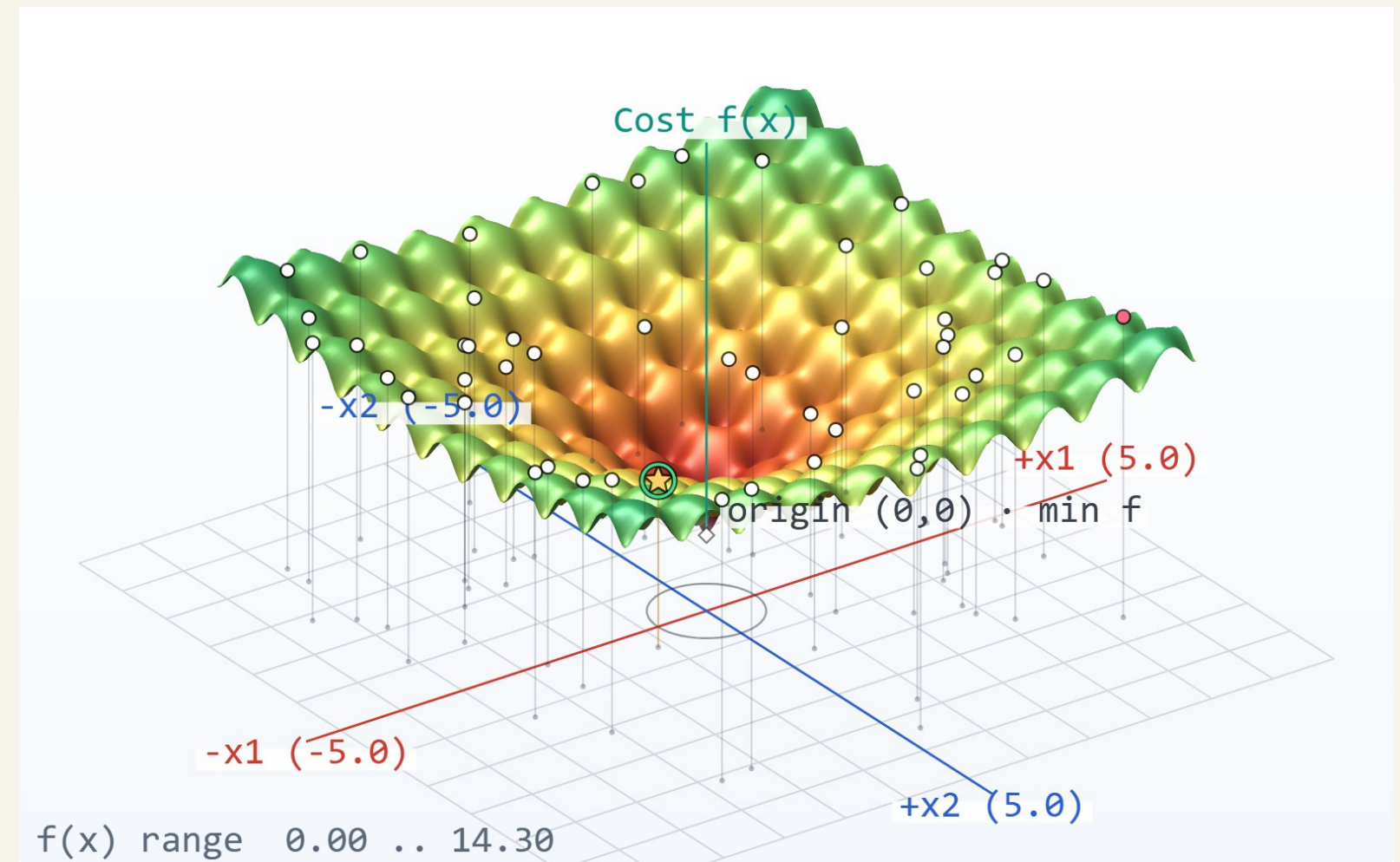


Fig. 01 — Ackley landscape. White-mode frontend; the central basin contains the known optimum.

The route through the project

Problem → design choices → a real run → evidence



01 The problem

GA basics and the Ackley landscape



02 The settings

Parameters and a tuning experiment



03 The methods

How each operator works—and why



04 The walkthrough

Seven steps in the white frontend



05 The evidence

Convergence, spikes, and final result



06 The takeaway

Conclusion and references

GA: improve a population through generations

A genetic algorithm searches by selecting, combining, and randomly changing candidate solutions.



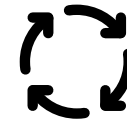
Population

Several candidate answers at once.



Selection

Better candidates get more chances.



Next generation

Keep useful changes; repeat.

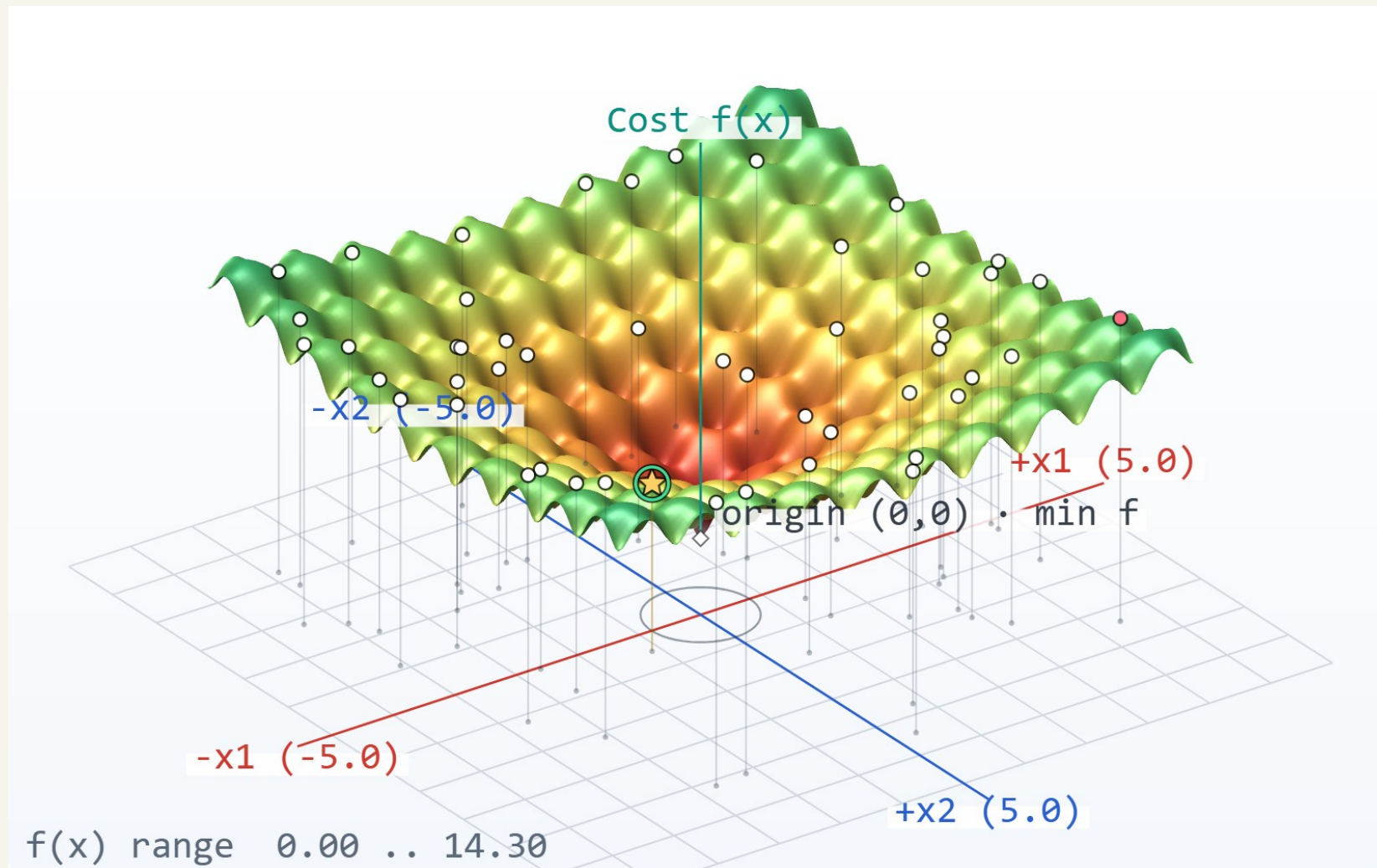
In this project

One chromosome = $[x_1, x_2]$ • One score = Ackley $f(x_1, x_2)$

Lower objective values are better.

The problem: find the bottom of a rippled basin

Minimize Ackley over $x_1, x_2 \in [-5, 5]$. The known global minimum is $f(0, 0) = 0$.



$$\min f(x_1, x_2)$$

$$\begin{aligned} & -20 \exp[-0.2 \sqrt{((x_1^2 + x_2^2)/2)}] \\ & - \exp[(\cos 2\pi x_1 + \cos 2\pi x_2)/2] \\ & + 20 + e \end{aligned}$$

Why this is challenging

Local valleys can trap the search before it reaches the central basin.

Fig. 02 — Search landscape. White-mode 3D view; many local basins surround the origin.

The baseline settings balance search and preservation

These are the project defaults used in every walkthrough and result slide.

50

individuals

Variety with a small evaluation budget.

0.80

crossover / pair

Recombine most pairs; keep some copies.

0.05

mutation / gene

About 5 changes among 100 genes.

1.0

Gaussian σ

Steps span 10% of the coordinate range.

1

elite

Protect progress while most slots evolve.

42

random seed

Replay the same random decisions.

Tuning: smaller steps help, but can miss the basin

20 runs per setting · same seeds 42–61 · change one parameter · default stopping rules

SETTING	MEDIAN FINAL $f(x)$ ↓		WORST RUN ↓
Baseline		0.0673	0.1775
$P_c = 0.70$		0.0458	0.2970
$P_m = 0.10$		0.0187	0.0758
$\sigma = 0.10$		0.0052	2.5802
$\sigma = 0.50$		0.0247	0.1387
$N = 100$		0.0264	0.0971

$\sigma = 0.10$: strong median, weak worst case.

Tune exploration and precision together.

Exploratory comparison; larger N uses more evaluations. No setting reached $f < 0.001$ in these 20 runs.

Initialization: store real numbers and spread the search

Each coordinate is sampled independently and uniformly from $[-5, 5]$.



Uniform initialization

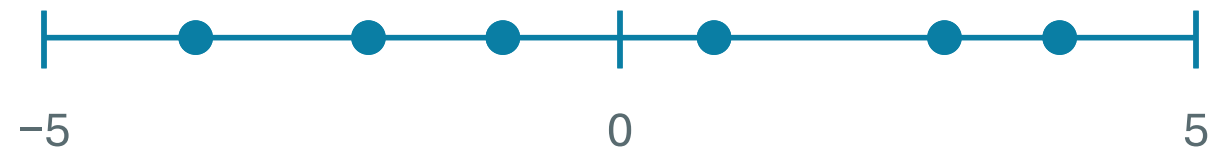
Equal sampling density across the domain.

Why: broad starting coverage, with no preferred region.

One chromosome



Each gene $\sim U(-5, 5)$



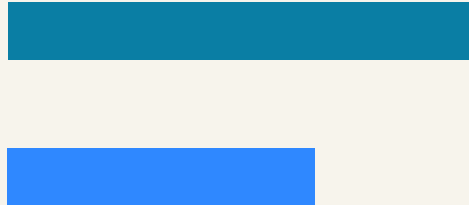
Selection: turn low cost into a higher chance

Fitness is recomputed relative to the worst individual in each generation.

$$F_i = f_{\text{worst}} - f_i$$

$$p_i = F_i / \Sigma F$$

Toy scores

A: $f = 2$		60%
B: $f = 4$		40%
C: $f = 8$		0%



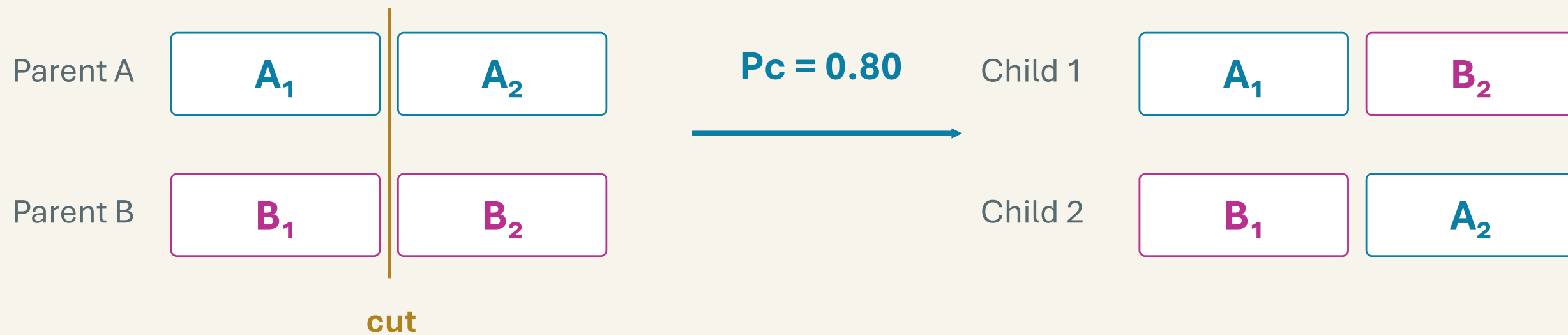
Roulette sampling

50 draws with replacement

Why: reward better candidates while keeping selection random.
If every fitness is zero, choose uniformly at random.

Crossover: swap the second coordinate

With two genes, there is exactly one cut point.



Why this method?

Simple coordinate recombination; preserves useful gene values.

What it cannot do

Create a new coordinate value.
That job belongs to mutation.

Mutation: introduce new coordinate values

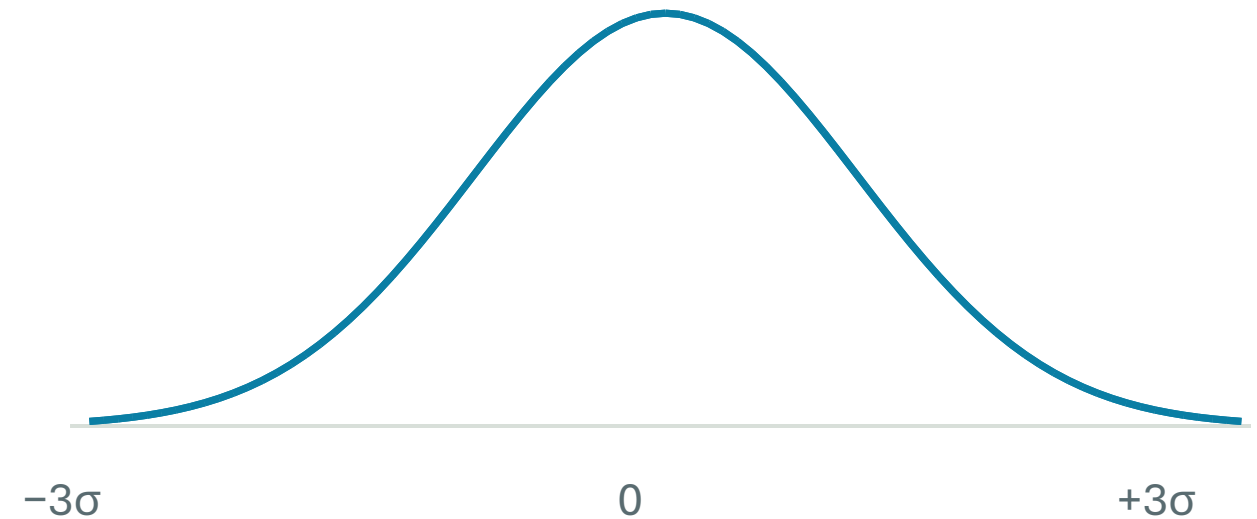
For each gene: mutate with $P_m = 0.05$; otherwise keep the original value.

$$\mathbf{x}' = \mathbf{x} + \varepsilon$$

$$\varepsilon \sim \text{Normal}(0, \sigma^2), \sigma = 1$$

Why Gaussian?

Explore around the current point;
occasionally take a larger step.



Out of bounds? Resample in $[-5, 5]$.

Protect the best—and know when to stop

Two stopping checks limit wasted work; neither proves the exact optimum has been found.



Keep one elite

Replace the worst child only if the saved elite is better.

Why: preserve progress.



100 generations

A fixed upper bound on the run length.

Why: control computation.



30 stagnant generations

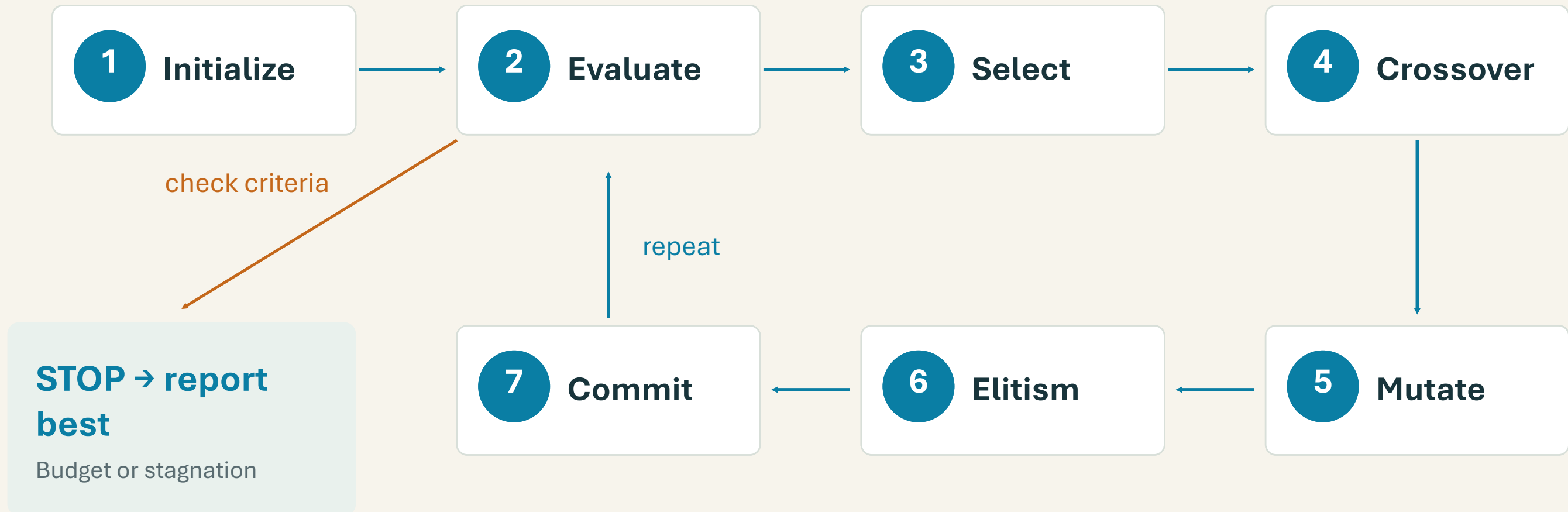
Stop if no improvement $> 10^{-12}$ persists for 30 generations.

Why: end an unproductive run.

$f(x) < 0.001$ is a reporting threshold in this project, not an early-stop rule.

The complete GA flow

Initialization happens once; the remaining stages repeat.



Evaluate → transform fitness → save elite → select → cross → mutate + repair → elitism → commit

1 • Initialize 50 candidate points

One run throughout: 50 individuals · P_c 0.80 · P_m 0.05 · σ 1.0

1 Init

2 Evaluate

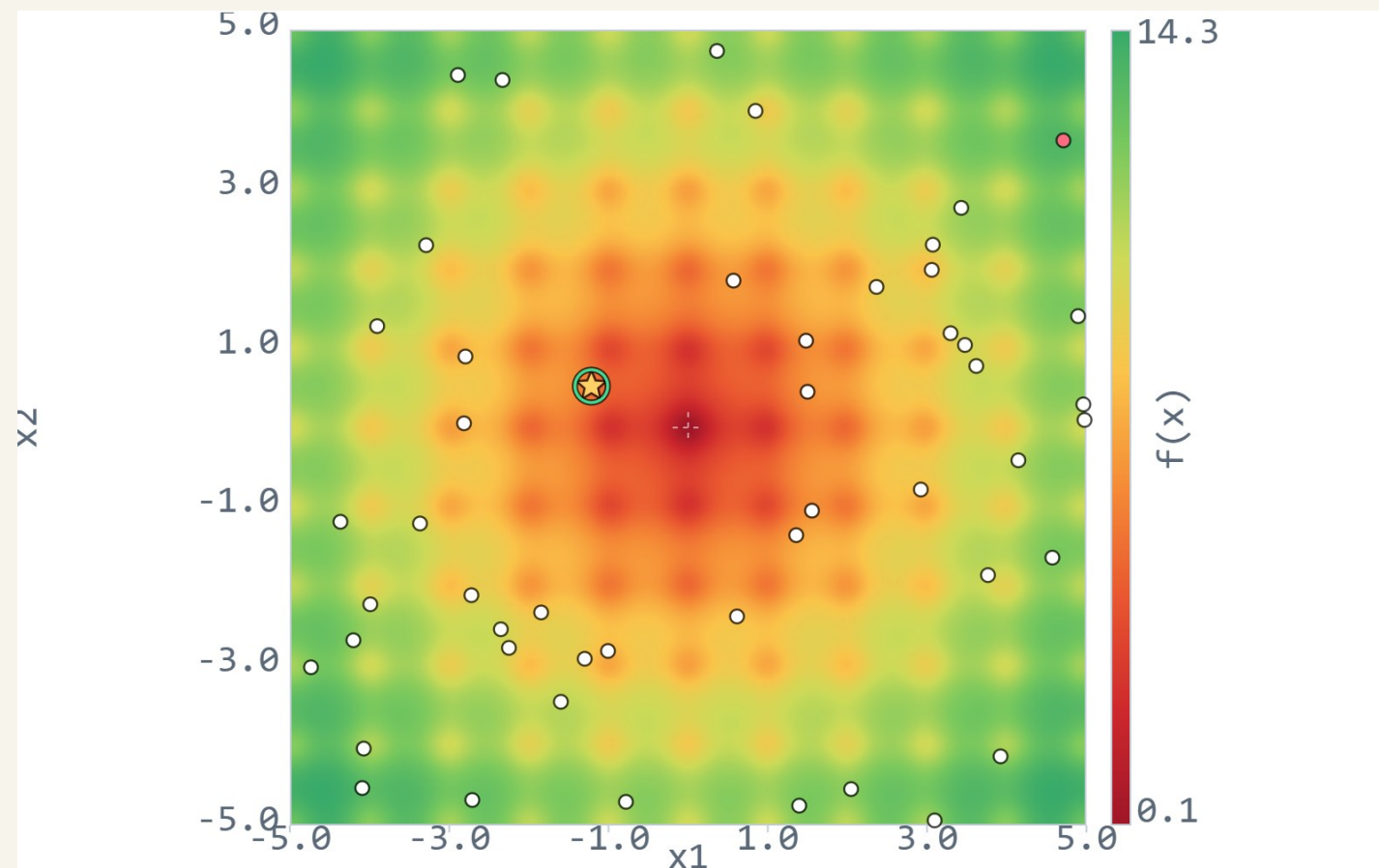
3 Select

4 Cross

5 Mutate

6 Elite

7 Commit



One point = one chromosome

The coordinates are the two genes.

Uniform random start

No candidate is placed at the origin by design.

Repeatable example

Seed 42 fixes the random stream.

Fig. 03 — Initialize 50 candidate points. Generation 1, seed 42; points cover the search square.

2 · Evaluate and rank the population

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

- 1 Init
- 2 Evaluate
- 3 Select
- 4 Cross
- 5 Mutate
- 6 Elite
- 7 Commit

1 Evaluate & rank

population 50

best f(x) 5.447495 average 9.9798 f_worst 13.4771 stagnation 0 / 30

#	x1	x2	f(x)	fitness	share
17	-1.215	0.520	5.4475	8.030	
7	1.499	0.449	6.3114	7.166	
37	1.554	-1.044	6.3605	7.117	
27	1.480	1.091	6.3721	7.105	
31	0.569	1.846	6.6554	6.822	

green bar

 elite (kept)

gold row

 best

red row

 worst, fitness 0

Best is individual #17

f = 5.4475 at (−1.2147, 0.5204).

Convert cost to fitness

13.4771 − 5.4475 = 8.0296.

Save the elite

Keep a protected copy before breeding.

Fig. 04 — Evaluate and rank the population. White-mode inspector; five highest-ranked rows shown.

3 · Select the mating pool

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

1 Init

2 Evaluate

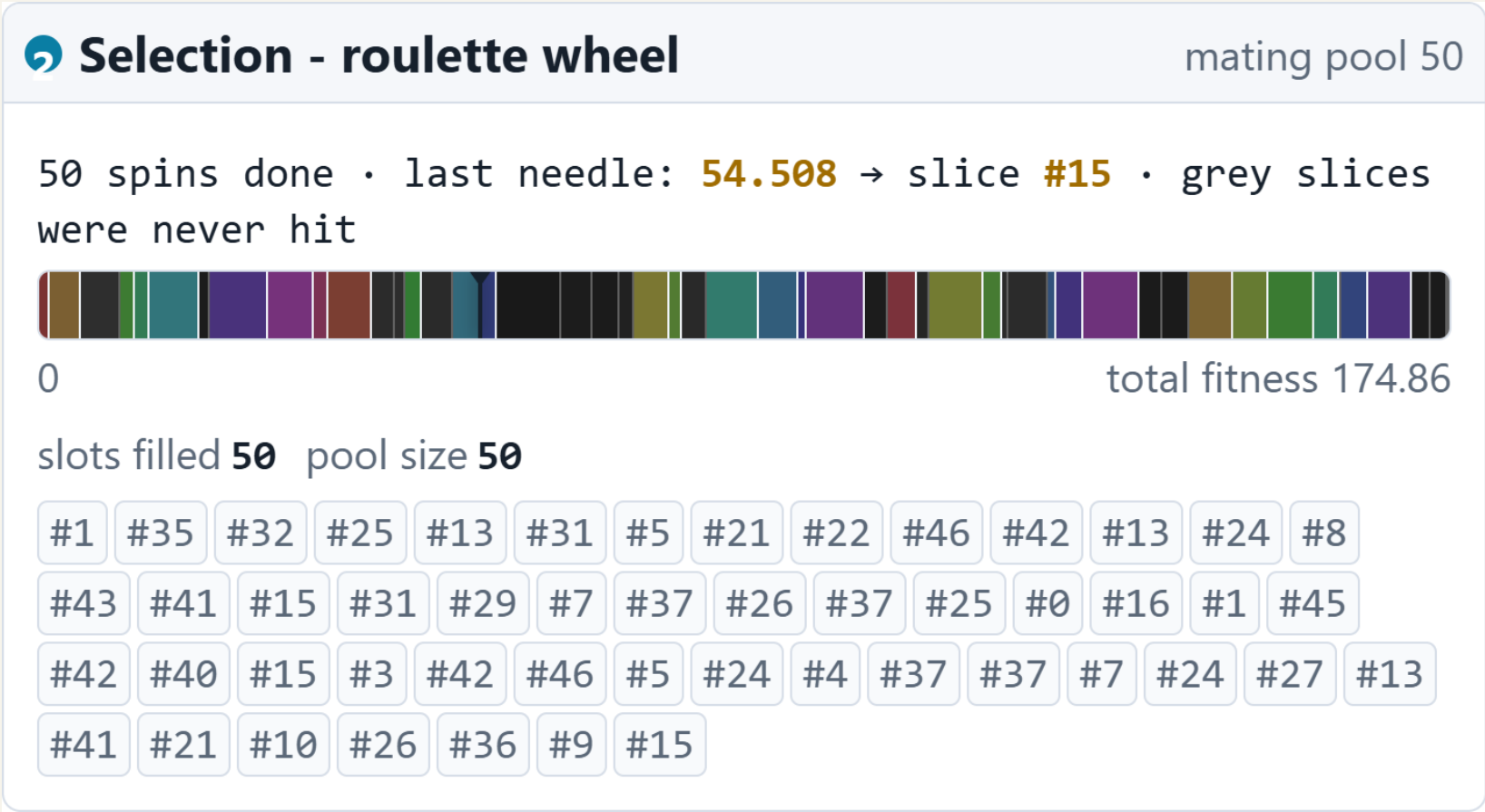
3 Select

4 Cross

5 Mutate

6 Elite

7 Commit



Draw 50 parent slots

The first four IDs: #1, #35, #32, #25.

Duplicates are allowed

#37 is selected four times.

Better is more likely

A strong candidate is favoured, not guaranteed.

Fig. 05 — Select the mating pool. White-mode inspector; roulette tape and complete mating pool.

4 · Crossover creates two children

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

1 Init

2 Evaluate

3 Select

4 Cross

5 Mutate

6 Elite

7 Commit

Crossover - 1-point

25 pairs

pair 2 · slots 2,3 (from #32, #25) $r = 0.6499 \leq 0.8 \rightarrow$ crossover at point 1

parent A

3.42852.7600

parent B

-1.2982-2.9049

↓ children

child 2

3.4285-2.9049

child 3

-1.29822.7600

The random roll is 0.6499
 $0.6499 \leq 0.80 \rightarrow$ crossover happens.

Cut between x_1 and x_2
Parents #32 and #25 swap their tails.

19 of 25 pairs cross
The expected count is 20; actual counts vary.

Fig. 06 — Crossover creates two children. White-mode inspector; pair 2 shown from 25 pairs.

5 • Mutate selected genes

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

1 Init

2 Evaluate

3 Select

4 Cross

5 Mutate

6 Elite

7 Commit

 Mutation - gaussian

5 of 100 genes

mutated genes 5 children 50

child 5

1.9640

+1.39

1.8461

0.236

child 8

-4.2021

0.165

0.1472

+0.93

child 23

-1.2982

0.579

0.2300

+1.27

5 of 100 genes mutate

Each coordinate has its own 5% chance.

Child 5, first coordinate

$0.5695 + 1.3945 = 1.9640.$

Repair only if needed

All five mutations stay within $[-5, 5]$.

Fig. 07 — Mutate selected genes. White-mode inspector; three changed children shown.

6 · Insert the protected elite

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

- 1 Init
- 2 Evaluate
- 3 Select
- 4 Cross
- 5 Mutate
- 6 Elite
- 7 Commit

5

Elitism

1 elite

elite 1 vs worst child #47

elite	-1.2147	0.5204	f = 5.44749
worst	3.7637	4.3665	f = 13.12051

replaced elite f = 5.44749 < worst f = 13.12051

Saved elite: f = 5.4475

The best parent was protected before breeding.

Worst child: f = 13.1205

Child #47 is replaced by the better elite.

Population stays at 50

One replacement; no extra individual.

Fig. 08 — Insert the protected elite. White-mode inspector; the actual generation-1 replacement.

7 · Commit and repeat

One run throughout: 50 individuals · Pc 0.80 · Pm 0.05 · σ 1.0

1 Init

2 Evaluate

3 Select

4 Cross

5 Mutate

6 Elite

7 Commit

New generation

generation 2

best before **5.447495** best now **2.980182** average now **8.6868**

best ever **5.447495**

improved stagnation counter 0 / 30

Children become generation 2

The best candidate now has $f = 2.9802$.

A better combination

New best: (0.8459, 0.0953).

Next: evaluate again

Update history, check stopping, and repeat.

Fig. 09 — Commit and repeat. White-mode generation summary after the first breeding cycle.

Watch the population gather around the origin

Animated white-mode frontend replay · selected evaluated generations from the same run

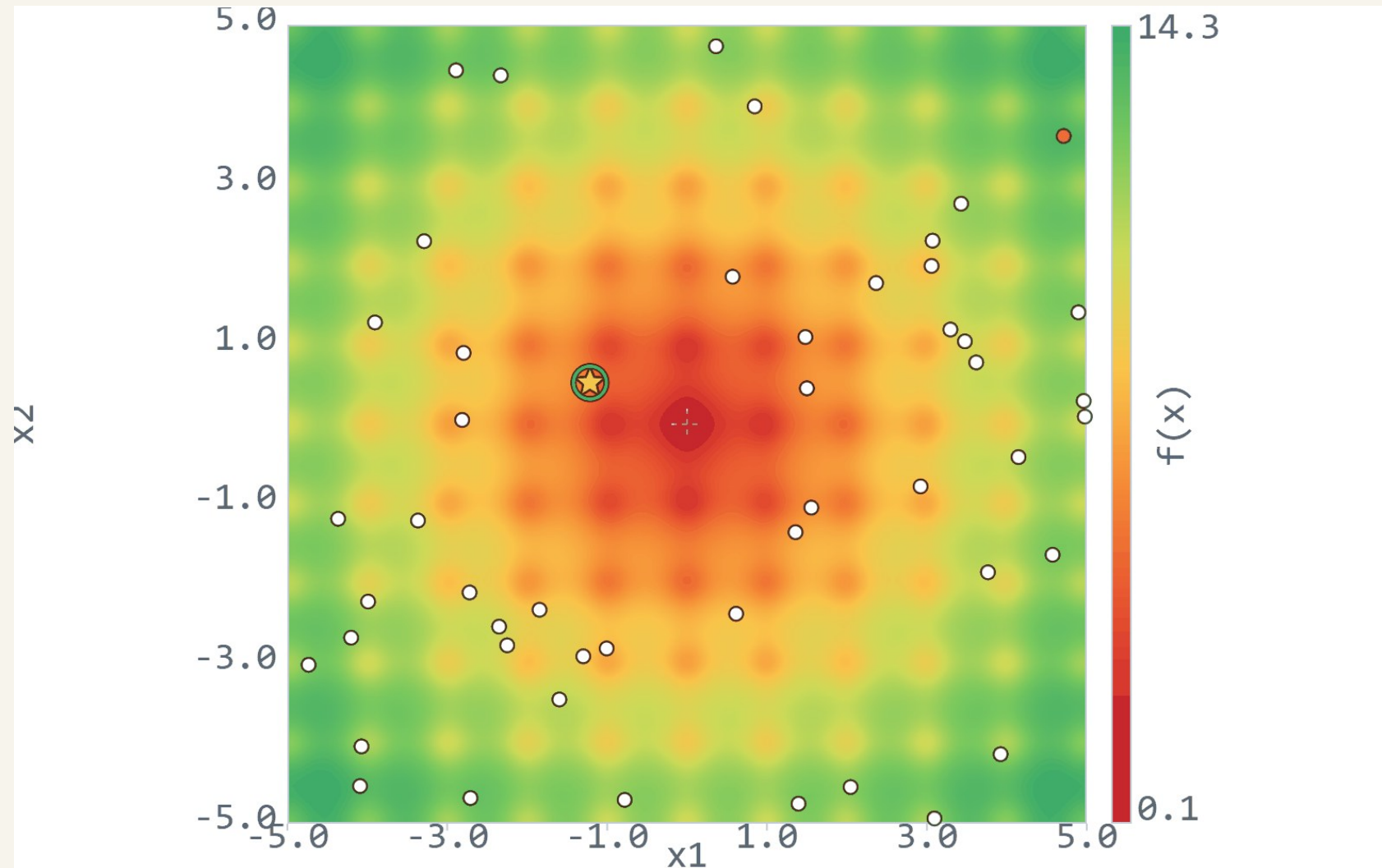


Fig. 10 — Population replay. 10 snapshots, generations 1–100; loops in PowerPoint Slide Show.

Early: broad movement

Selection and recombination rapidly reduce the objective.

Later: a compact cluster

New mutations still send some points away from the centre.

A persistent best point

Elitism keeps the strongest candidate safe.

Convergence: fast gains, long plateaus, a late drop

Blue = best objective · orange = population average · logarithmic vertical scale

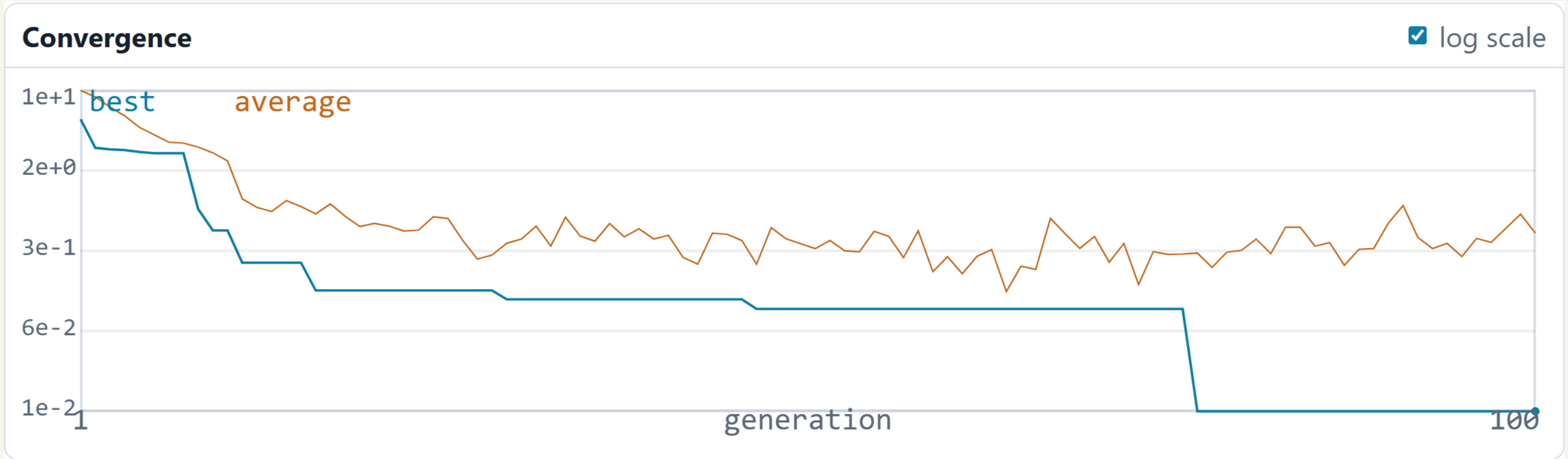


Fig. 11 — Convergence curve. Actual white-mode frontend; 100 evaluated generations, seed 42.

G1 → G5

5.4475 → 2.7318

G47 → G76

Best stays at 0.10067

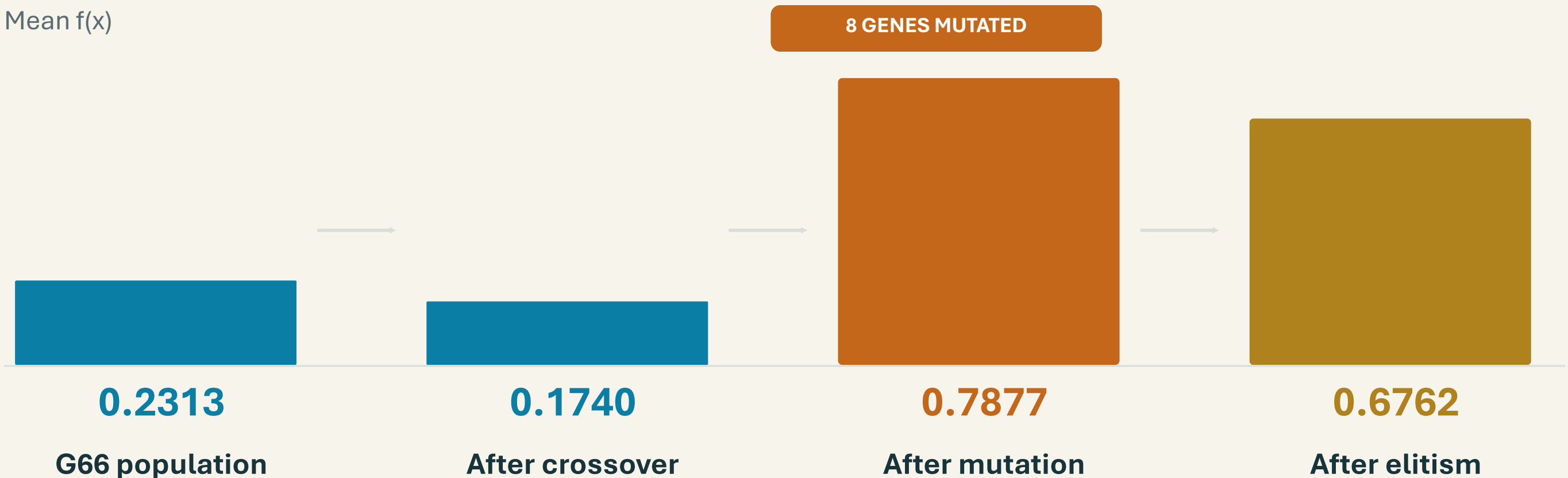
G77 → G100

Best reaches 0.011695

Why did the average spike at generation 67?

The generation-66 mutation stage produced the population evaluated at generation 67.

Mean $f(x)$



Best stays at 0.10067. Mutation worsened the mean; elitism removed the worst child.

Final result: close to the known optimum

The run ends at the 100-generation limit, with 23 consecutive stagnant generations.

Best point

(0.003973, 0.000317)

$f(x) = 0.011695$

Distance to origin: 0.003985

99.79%

reduction from the initial best

5.447495 → 0.011695

Near the centre; still above 0.001.

The known answer is (0, 0), with $f = 0$.

Why did the run stop short of zero?

The final point is good, but the operators make very fine improvement increasingly unlikely.



Limited recombination

One-point crossover can only reuse existing coordinates.



A fixed step size

$\sigma = 1$ is large compared with the final ≈ 0.004 distance.



A finite search budget

No better candidate appears after generation 77.

A useful next experiment

Reduce σ later in the run, then test across many seeds.

The GA found the right basin—and exposed its limits

A visual run makes both the progress and the remaining gap explainable.

01

Selection directs the search

Fitter candidates receive more opportunities.

02

Variation explores; elitism protects

Average quality can fall while the best is preserved.

03

Tuning needs repeated evidence

Check typical quality, poor runs, and evaluation cost.

FINAL BEST 0.011695

Near the optimum; further precision needs better search control.

References and reproducibility

Primary sources, local implementation, and the evidence used in this presentation

01 Project implementation

`ga_ackley.py`; `ga-web/engine.py`; `ga-web/static/`

Exact operators, parameters, stage traces, and white-mode captures.

02 Ackley benchmark

[Surjanovic & Bingham · SFU Virtual Library](#)

Definition, landscape, recommended constants, and known optimum.

03 Genetic algorithms

[John H. Holland · Adaptation in Natural and Artificial Systems](#)

MIT Press, 1992 edition; foundations of adaptation and genetic search.

04 Icons

[ByteDance IconPark + the supplied custom icons](#)

SVG assets from icons/; IconPark under Apache 2.0.

05 Run evidence

`presentation/data/run.json` and `summary.json`

Seed-42 trace; 180 tuning runs; all 14 existing parity checks passed.

Web sources accessed 12 September 2026 · full links and run details are also in speaker notes.